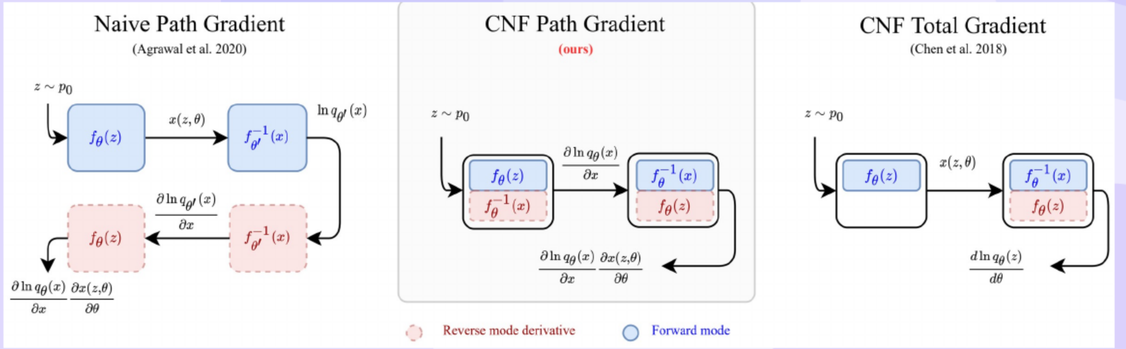
Path-Gradient Estimators for Continuous Normalizing Flows

Lorenz Vaitl; Kim Nicoli; Shinichi Nakajima; Pan Kessel

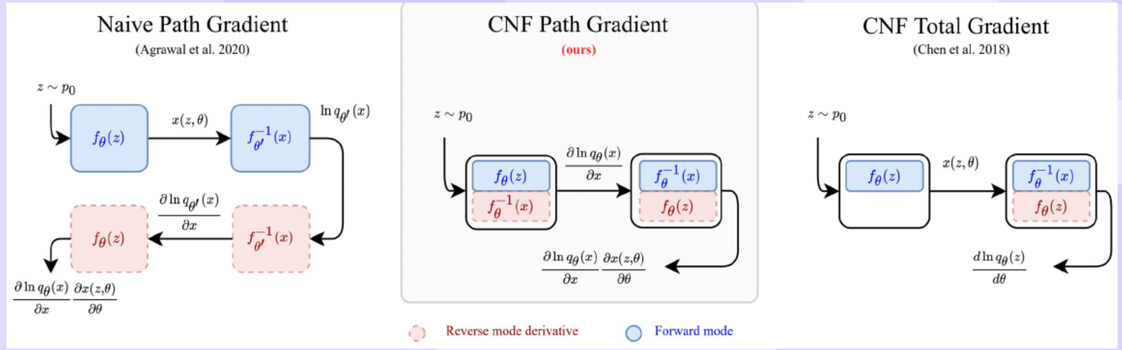
01 Motivation and Contributions



- Lower variance training with **constant memory**
- Approximately four forward solves versus six for cloning
- Faster convergence across variational inference tasks

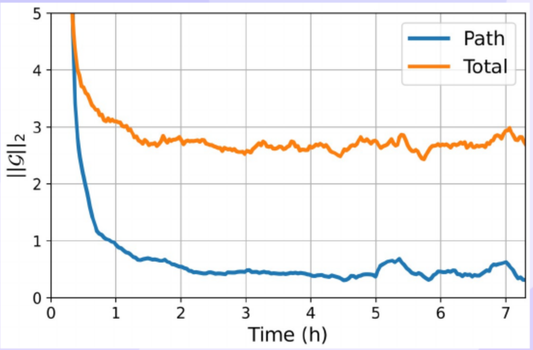
- Simple drop in replacement for **total gradients**
- Figure: compares naive, our path, and total gradients.

02 Related Work



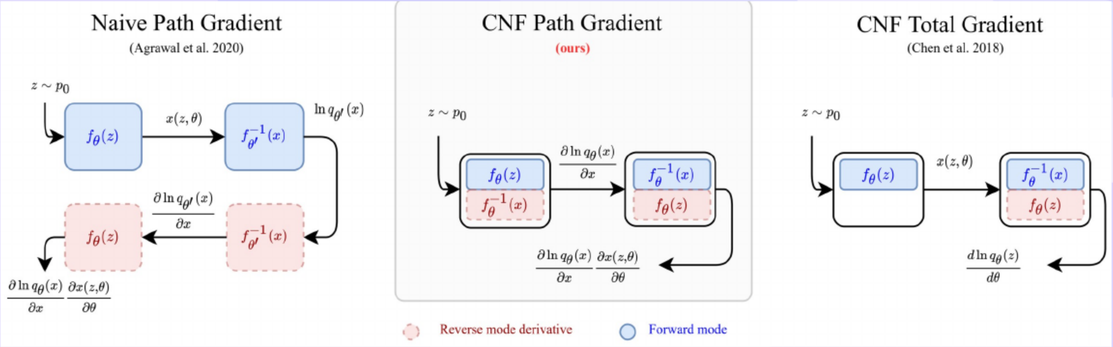
- Path gradients extended to VAE losses and alpha divergence variants
- Prior flow training relied on model cloning
- Our approach avoids duplication and scales efficiently
- Figure: cloning duplicates models; ours avoids it.

03 VI and Gradient Split



- Reverse divergence gradient splits into path and score terms
- Score term has zero mean and equals **Fisher information** in variance
- Near matching distributions, path term shrinks and variance drops
- Figure shows lower norm and faster decay for path gradients.

04 Why Naive Path Fails

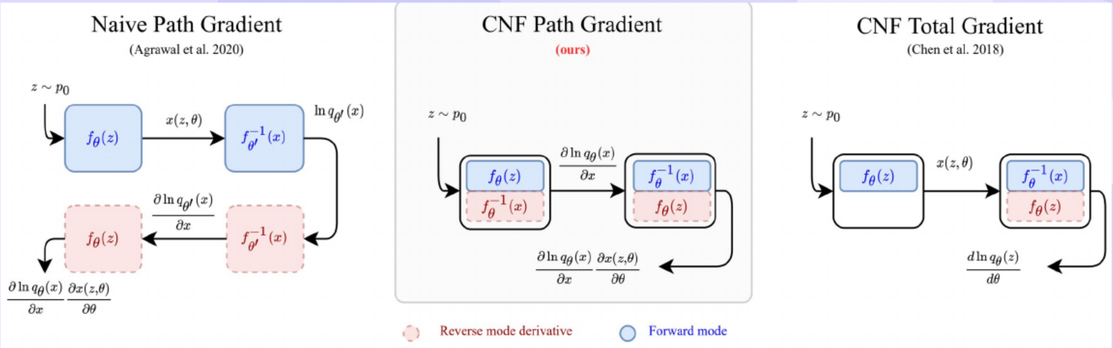


- Sampling and log density computation are tightly coupled
- Naive parameter swap forces Jacobian reuse and cloning
- Cloning doubles memory and inflates runtime substantially
- Figure: Coupling forces cloning and extra cost.

05 CNF Basics

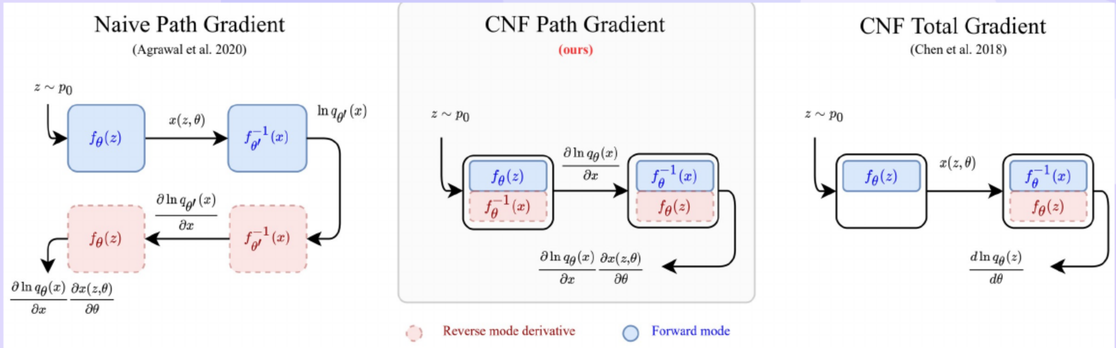
- Continuous flows transform base noise via an ordinary differential equation
- Log density accumulates divergence along the trajectory
- Hutchinson estimator provides unbiased trace estimation

06 Adjoint Baseline



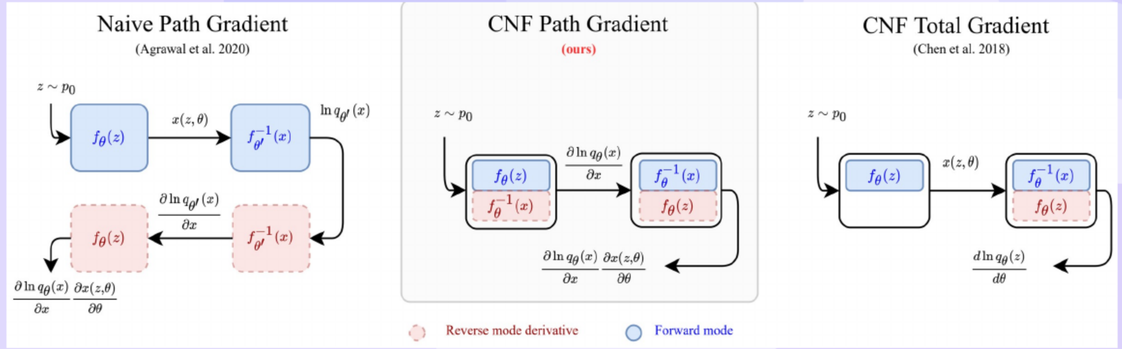
- Forward solve computes terminal state without storing activations
- Backward adjoint recomputes states for vector products
- Total cost is about three forward equivalent solves
- Figure: Adjoint recomputes states; total cost ≈3 solves with constant memory.

07 Our CNF Path-Gradient



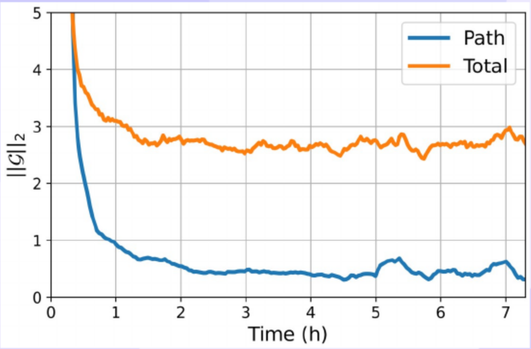
- Forward initial value problem computes gradient with respect to state
- Adjoint applies vector Jacobian product to parameters
- Overall cost is about four forward equivalent solves with **constant memory**
- Figure: Forward-IVP plus adjoint; cost ≈4 solves, still constant memory.

08 Complexity vs Baselines



- Adjoint total derivative uses about three forward solves
- Our method uses about four solves with shared Jacobian vector products
- Cloning needs about six solves and double memory
- Figure: ours \u22484 solves, cloning \u2486 and \u2482\u00d7 memory.

09 Variance: Intuition, Caveats

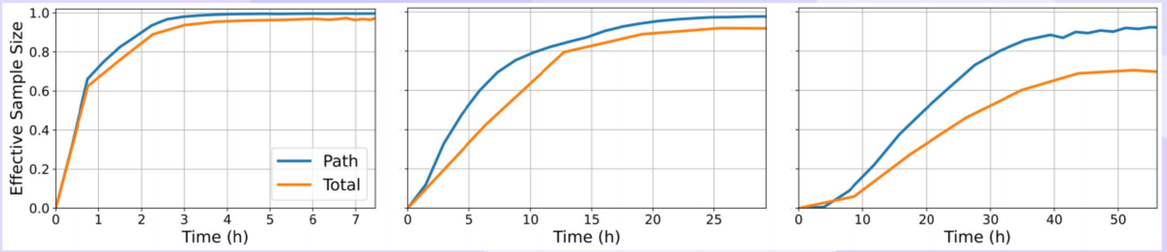


- Removing the score term eliminates variance from **Fisher information**
- Variance reduction strengthens near convergence
- Early training may exhibit large path variance and instability
- Figure: Variance drops near convergence; caveats remain.

10 Tasks and Setup

- Lattice models with symmetry invariant continuous flows
- Variational autoencoders with continuous flows on four datasets
- Reverse divergence objective with matched hyperparameters and hardware

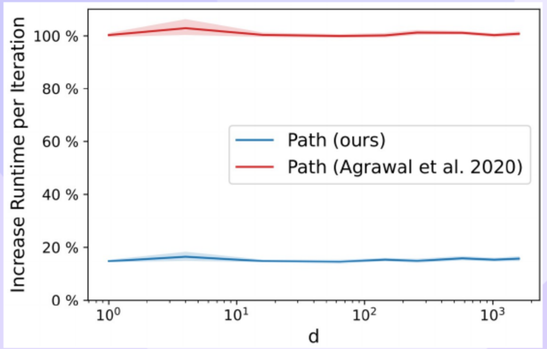
11 Results Summary



- Main message: Path-gradients reach better final metrics faster with modest overhead.
- Higher effective sample size and faster convergence on lattices
- Improved evidence bounds on multiple vision datasets

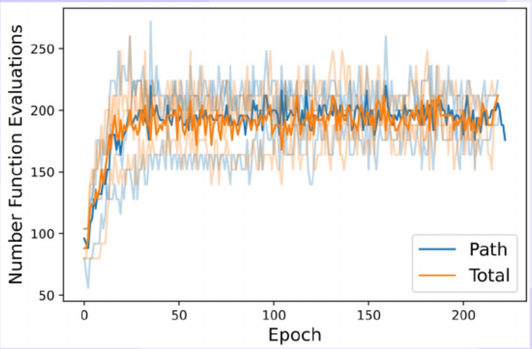
- Curves: ESS vs time—path lines dominate across lattices.
- Across lattices, the blue path curve reaches higher ESS earlier.

12 Practical Notes



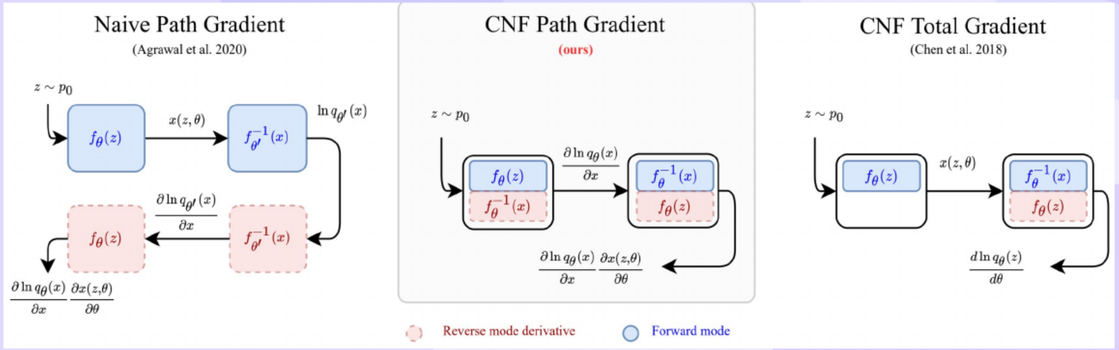
- Wall time overhead stays below cloning and remains practical
- Memory usage matches the adjoint baseline
- Method works with fixed step and adaptive solvers
- Runtime increase vs dimension—ours remains low.

13 Implementation Tips



- Reuse Hutchinson probes and align random number streams
- Match solver tolerances and mirror event handling in adjoint
- Clip divergences, share Jacobian vector products, and cache products
- Figure: Path uses fewer function evaluations on average.

14 Conclusion and Limits



- Path gradients deliver constant memory with modest overhead
- Empirical gains appear across continuous flow variational tasks
- Theory is strongest near convergence and early phase remains open
- Figure: summarizes constant memory gains and open early-phase theory.

15 Key Takeaway

- Path gradients enable lower variance constant memory training with better variational performance and modest runtime cost
- The method is a practical drop in replacement for standard gradients
- The approach avoids model cloning and scales efficiently

Thank You

Questions & Discussion